1. What is a test, and why do we use JUnit?

“Testing is the process of checking whether our application works as expected. In Java, we write test cases to verify that our methods produce the correct output for different inputs.

JUnit is a testing framework for Java that helps us write and execute tests automatically. It provides annotations to identify test methods and assertions to check whether the actual output matches the expected output.

We use JUnit to find bugs early, reduce manual testing, and make sure that new code changes do not break existing functionality. This is especially useful in real projects, where developers frequently modify code.”

Real-life example

Imagine we are developing an e-commerce website. When a customer purchases a product worth ₹1,000 with a 10% discount, the final price should be ₹900.

Instead of manually checking this calculation every time we change the code, we can write a JUnit test that checks the result automatically.

Java code

First, write the business logic:

```
DiscountCalculator.java

package com.example;

public class DiscountCalculator {

    public double calculateFinalPrice(
            double price, double discountPercent) {

        if (price < 0) {
            throw new IllegalArgumentException(
                "Price cannot be negative");
        }

        if (discountPercent < 0 || discountPercent > 100) {
            throw new IllegalArgumentException(
                "Invalid discount percentage");
        }

        return price - (price * discountPercent / 100);
    }
}
```

Now write the JUnit test:

```
DiscountCalculatorTest.java

package com.example;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class DiscountCalculatorTest {

    private final DiscountCalculator calculator =
```

```
            new DiscountCalculator();

    @Test
    void shouldCalculateFinalPriceAfterDiscount() {

        double actual =
                calculator.calculateFinalPrice(1000, 10);

        assertEquals(900.0, actual, 0.001);
    }

    @Test
    void shouldRejectDiscountAbove100Percent() {

        assertThrows(
            IllegalArgumentException.class,
            () -> calculator.calculateFinalPrice(1000, 150)
        );
    }
}
```

How to explain the code in an interview:

“In the first test, I pass a price of ₹1,000 and a discount of 10%. The expected result is ₹900, so I use `assertEquals()` to compare the expected and actual results.

In the second test, I pass an invalid discount of 150%. I use `assertThrows()` to verify that the method throws an `IllegalArgumentException`.

This way, I test both normal behaviour and invalid input automatically.” JUnit Jupiter provides test annotations and assertions for these purposes. 【turn326473view0】

Likely follow-up questions

Q. What is unit testing?

“Unit testing means testing a small, individual part of an application, such as a method or class, separately from other parts.”

Q. What is the difference between manual testing and automated testing?

“Manual testing requires a person to execute test steps, whereas automated testing uses code or tools to execute predefined tests and check the results.”

Q. Does JUnit test the entire application automatically?

“No. We have to write the test cases ourselves. JUnit provides the framework to execute them and check whether they pass or fail.”

2. What is Maven?

Interview-ready answer

“Maven is a build automation and project management tool mainly used in Java projects.

It helps us manage project dependencies, compile source code, run tests, package the application into a JAR or WAR file, and manage the overall build process.

Maven uses a configuration file called `pom.xml`, which stands for Project Object Model. In this file, we define project information, dependencies, and build configurations.

For example, if I want to use JUnit in my Java project, I can add the JUnit dependency in `pom.xml`. Maven downloads the required library and makes it available to the project, so I don't have to manage the library manually."

Maven's standard project structure separates application code from test code, and its build lifecycle coordinates tasks such as compilation and testing. 【turn116048search0】【turn326473view1】

Real-life example

Imagine you are working on a Java project with five developers. Everyone needs the same JUnit library to run the tests.

Without dependency management, team members might have to download and configure libraries separately. With Maven, we declare the dependency in `pom.xml`, and Maven resolves it for the project.

Important Maven configuration

Add the following inside your project's `pom.xml`:

```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.13.1</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.5.3</version>
        </plugin>
    </plugins>
</build>
```

These are example versions for a JUnit 5 project, not a claim that they are the latest releases.

What does this configuration mean?

- `groupId` and `artifactId` identify the JUnit library.
- `version` specifies the library version.
- `scope=test` means the dependency is intended for testing and isn't required as a normal runtime dependency of the packaged application.
- `maven-surefire-plugin` helps Maven discover and execute unit tests during the test phase.

Standard Maven folder structure

```
my-project/
│
├── pom.xml
└── src/
    ├── main/
    │   └── java/
```

```
│       └── com/example/
│           └── DiscountCalculator.java
│
└── test/
    └── java/
        └── com/example/
            └── DiscountCalculatorTest.java
```

Important commands to remember

Command	Purpose
`mvn clean`	Removes generated build files.
`mvn compile`	Compiles the main source code.
`mvn test`	Compiles test code and runs tests.
`mvn package`	Packages the application after the preceding required phases.
`mvn install`	Installs the built artifact in the local Maven repository.

Likely follow-up questions

Q. What is `pom.xml`?

"`pom.xml` is Maven's main project configuration file. It contains project details, dependencies, and build configuration."

Q. What is a dependency in Maven?

"A dependency is an external library that our project needs. For example, JUnit is a dependency used for testing Java code."

Q. How do you run JUnit tests using Maven?

"I open the terminal in the directory containing `pom.xml` and run `mvn test`. Maven builds the required parts of the project and executes the configured tests." 【turn116048search1】【turn326473view1】

3. What are assertions and annotations in JUnit 5?

This question has two separate concepts, so explain them separately in the interview.

A. Assertions

Interview-ready answer

"Assertions are methods used to verify whether the actual result of a test matches the expected result. If the assertion succeeds, that check passes. If it fails, JUnit marks the test as failed and reports the mismatch."

Think of an assertion as a quality check: Did the application produce the result we expected?

Common assertion methods

Assertion	Purpose
`assertEquals(expected, actual)`	Checks whether two values are equal.
`assertTrue(condition)`	Checks whether a condition is true.
`assertFalse(condition)`	Checks whether a condition is false.

`assertNull(value)`	Checks whether a value is `null`.
`assertNotNull(value)`	Checks whether a value is not `null`.
`assertThrows(Exception.class, ...)`	Checks whether the expected exception is thrown.
`assertAll(...)`	Executes multiple assertion checks in one grouped test.

These are provided by JUnit's `Assertions` API. 【turn326473view0】

Example

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class AssertionTest {

    @Test
    void testAssertions() {

        assertEquals(900.0, 900.0, 0.001);

        assertTrue(10 > 5);

        assertFalse(10 < 5);

        assertNotNull("JUnit");

        assertNull(null);
    }
}
```

How to explain it:

“Here, I verify equality, true and false conditions, and null values. JUnit reports the test as failed if any assertion fails.”

B. Annotations

Interview-ready answer

“Annotations are metadata that tell JUnit how to identify and execute test-related methods. We use annotations such as `@Test`, `@BeforeEach`, and `@AfterEach` to mark test methods and control the test lifecycle.”

Common JUnit 5 annotations

Annotation	Meaning
`@Test`	Marks a method as a test method.
`@BeforeEach`	Runs before each test method.
`@AfterEach`	Runs after each test method.
`@BeforeAll`	Runs once before all test methods in the class.
`@AfterAll`	Runs once after all test methods in the class.
`@Disabled`	Prevents a test from running.
`@DisplayName`	Gives a test or test class a readable name.
`@ParameterizedTest`	Runs a test with multiple sets of arguments.

【turn326473view0】

Example

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.DisplayName;

import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculatorTest {

    @Test
    @DisplayName("Add two numbers")
    void shouldAddNumbers() {

        int result = 10 + 20;

        assertEquals(30, result);
    }
}
```

How to explain it:

“Here, `@Test` tells JUnit that `shouldAddNumbers()` is a test method. `@DisplayName` gives the test a human-readable name, and `assertEquals()` checks the result.”

Most important follow-up question

Q. What is the difference between an assertion and an annotation?

“An annotation tells JUnit what a method is or when it should run, while an assertion checks whether the program's behaviour or output is correct.”

Remember this distinction:

- Annotation = identifies or configures the test.
- Assertion = checks the result of the test.

4. Explain the JUnit 5 test lifecycle.

Interview-ready answer

“The JUnit test lifecycle is the sequence of steps performed before, during, and after test execution.

JUnit 5 provides lifecycle annotations that help us initialize test objects, prepare test data, execute test methods, and perform cleanup.

The main lifecycle annotations are `@BeforeAll`, `@BeforeEach`, `@Test`, `@AfterEach`, and `@AfterAll`.

For example, if I am testing a login service, I can use `@BeforeEach` to prepare the test data before every test and `@AfterEach` to clean up any resources used by that test.”

Test lifecycle order

1. `@BeforeAll`

Runs once before all tests in the class.

2. `@BeforeEach`

Prepares the test before each test method.

3. `@Test`

Executes the test logic and assertions.

4. `@AfterEach`

Cleans up after each test method.

5. `@AfterAll`

Runs once after all tests in the class.

Steps 2–4 repeat for each test method. The order in which multiple test methods run is not guaranteed by default, so tests should not depend on a particular order. 【turn326473view0】

Java code example

```
import org.junit.jupiter.api.*;

class LifecycleTest {

    @BeforeAll
    static void beforeAll() {
        System.out.println("Before all tests");
    }

    @BeforeEach
    void beforeEach() {
        System.out.println("Before each test");
    }

    @Test
    void testOne() {
        System.out.println("Executing test one");
    }

    @Test
    void testTwo() {
        System.out.println("Executing test two");
    }

    @AfterEach
    void afterEach() {
        System.out.println("After each test");
    }

    @AfterAll
    static void afterAll() {
        System.out.println("After all tests");
    }
}
```

What will happen when this code runs?

Conceptually, the output follows this sequence:

```
Before all tests
```

```
Before each test
Executing test one
After each test

Before each test
Executing test two
After each test

After all tests
```

The two test methods could execute in the opposite order; their default order is not guaranteed.

Important interview detail: In JUnit 5, `@BeforeAll` and `@AfterAll` methods are normally `static`. They can be non-static when the test class uses `@TestInstance(TestInstance.Lifecycle.PER_CLASS)`.

Real-life example

Suppose we are testing an online shopping application:

- `@BeforeAll`: Initialize expensive shared test infrastructure.
- `@BeforeEach`: Prepare a fresh shopping cart for a test.
- `@Test`: Check adding a product to the cart.
- `@AfterEach`: Clear test data or close resources.
- `@AfterAll`: Shut down shared test infrastructure.

The exact setup depends on the application; for example, database cleanup belongs in the appropriate cleanup method only when the test actually uses a database.

Likely follow-up questions

Q. What is the difference between `@BeforeAll` and `@BeforeEach`?

"`@BeforeAll` runs once for the whole test class, whereas `@BeforeEach` runs before every test method."

Q. Why do we use `@AfterEach`?

"To clean up test-specific state and resources, which helps prevent one test from affecting another."

5. Explain the architecture of JUnit 5.

Interview-ready answer

"JUnit 5 architecture consists of three main components: JUnit Platform, JUnit Jupiter, and JUnit Vintage.

The JUnit Platform provides the foundation for discovering and launching tests. JUnit Jupiter provides the programming model, annotations, and test engine used to write and execute JUnit 5-style tests. JUnit Vintage provides an engine for running older JUnit 3 and JUnit 4 tests on the JUnit Platform.

Together, these components allow modern and legacy tests to run through a common testing platform." 【turn326473view0】

Understand the three components

JUnit Platform

Foundation for discovering, launching, and executing tests through test engines.

JUnit Jupiter

Modern JUnit 5-style tests, annotations, assertions, and the Jupiter test engine.

JUnit Vintage

A test engine for legacy JUnit 3 and JUnit 4 tests.

Both engines can be used through the Platform when the necessary dependencies are present.

1. JUnit Platform

“The Platform is the foundation that test-launching tools use to discover and run supported tests. For example, Maven or an IDE can use the Platform to execute tests.”

2. JUnit Jupiter

“Jupiter is what I mainly use when writing tests with JUnit 5. It provides annotations such as `@Test`, lifecycle annotations, and a test engine that executes those tests.”

Example:

```
@Test
void shouldReturnCorrectPrice() {
    assertEquals(900.0, 900.0, 0.001);
}
```

This uses the Jupiter programming model and assertion API.

3. JUnit Vintage

“Vintage is useful when an existing project has older JUnit 3 or JUnit 4 tests that need to run on the JUnit Platform. We need the appropriate Vintage and legacy JUnit dependencies for that.”

How do they work together in a real project?

For example, when you execute `mvn test` in a configured Maven project:

1. Maven's test plugin discovers and launches the configured tests.
2. The JUnit Platform provides the test execution foundation.
3. The Jupiter engine executes JUnit 5-style tests.
4. If Vintage is configured, its engine can execute compatible JUnit 3 or JUnit 4 tests as well.
5. Maven reports the results, including failures.

Remember the architecture as:

- Platform: Launching and execution foundation.
- Jupiter: Writing and running modern JUnit tests.
- Vintage: Running legacy JUnit 3 and JUnit 4 tests.

Final interview revision

Question	One-line answer to remember
What is testing, and why JUnit?	Testing checks expected behaviour; JUnit automates Java test execution and result verification.
What is Maven?	A Java build automation and project management tool that manages dependencies and build tasks.
What are assertions?	Methods that verify test results and expected behaviour.
What are annotations?	Metadata that identifies or configures test methods and lifecycle behaviour.
What is the test lifecycle?	Setup, test execution, and cleanup managed through lifecycle annotations.
What is JUnit 5 architecture?	JUnit Platform + JUnit Jupiter + JUnit Vintage.

My interview tip: Learn the spoken answer first, then practise the `DiscountCalculator` example without looking at the code. If the interviewer asks you to demonstrate JUnit, explain `@Test`, `assertEquals()`, and `assertThrows()` while writing the test. These demonstrate both the concept and practical usage.

For reference, use the official JUnit 5 User Guide and Apache Maven Getting Started Guide.

JUnit 5 Interview Questions — To the Point

1. What is testing, and why do we use JUnit?

Interview answer:

"Testing means checking whether our code works as expected. JUnit is a Java testing framework that helps us test our code automatically, find bugs early, and ensure that changes do not break existing functionality."

Real-life example: In an online shopping application, a ₹1,000 product with a 10% discount should cost ₹900.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class DiscountTest {

    @Test
    void testDiscount() {
        double finalPrice = 1000 - (1000 * 10 / 100.0);

        assertEquals(900.0, finalPrice, 0.001);
    }
}
```

Explain the code: `@Test` marks the test method, and `assertEquals()` checks whether the actual result matches the expected result.

2. What is Maven?

Interview answer:

"Maven is a Java build automation and project management tool. It manages dependencies, compiles code, runs tests, and packages applications. We configure Maven using the `pom.xml` file."

Real-life example: Instead of manually downloading the JUnit library, we add its dependency to `pom.xml`, and Maven manages it for our project.

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.13.1</version>
    <scope>test</scope>
</dependency>
```

Run tests using:

```
mvn test
```

3. What are assertions and annotations in JUnit?

Interview answer:

"Annotations tell JUnit which methods to execute and when to execute them. Assertions check whether the actual result matches the expected result."

Common annotations: `@Test`, `@BeforeEach`, `@AfterEach`, `@BeforeAll`, `@AfterAll`.

Common assertions: `assertEquals()`, `assertTrue()`, `assertFalse()`, `assertNotNull()`, `assertThrows()`.

Code example:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class LoginTest {

    @Test
    void testLogin() {
        boolean loginSuccessful = true;

        assertTrue(loginSuccessful);
        assertEquals(true, loginSuccessful);
    }
}
```

Real-life example: In a login application, `@Test` marks the login test, and `assertTrue()` checks whether login succeeds.

Remember: Annotation = identifies/configures a test; assertion = checks the result.

4. Explain the JUnit test lifecycle.

Interview answer:

"The JUnit test lifecycle defines the sequence of setup, test execution, and cleanup. `@BeforeAll` runs once before all tests, `@BeforeEach` runs before every test, `@Test` executes the test, `@AfterEach` runs after each test, and `@AfterAll` runs once after all tests."

Code example:

```
import org.junit.jupiter.api.*;
```

```
class LifecycleTest {

    @BeforeAll
    static void setupAll() {
        System.out.println("Start testing");
    }

    @BeforeEach
    void setup() {
        System.out.println("Prepare test data");
    }

    @Test
    void testOne() {
        System.out.println("Run test");
    }

    @AfterEach
    void cleanup() {
        System.out.println("Clean test data");
    }

    @AfterAll
    static void finishAll() {
        System.out.println("Testing completed");
    }
}
```

Real-life example: When testing a shopping cart, we prepare a fresh cart before each test and clean it up afterward.

Order to remember:

`@BeforeAll` → `@BeforeEach` → `@Test` → `@AfterEach` → `@AfterAll`

5. Explain JUnit 5 architecture.

Interview answer:

“JUnit 5 has three main components: JUnit Platform, JUnit Jupiter, and JUnit Vintage.

- JUnit Platform: Provides the foundation for discovering and launching tests.
- JUnit Jupiter: Provides the programming model, annotations, assertions, and test engine for modern JUnit tests.
- JUnit Vintage: Supports running older JUnit 3 and JUnit 4 tests on the Platform.”

Real-life example: When I run tests through Maven, the Platform provides the execution foundation, Jupiter runs my JUnit 5 tests, and Vintage can run legacy tests if configured.

Remember: Platform = foundation; Jupiter = modern tests; Vintage = legacy tests.

Official reference: JUnit 5 User Guide.

Maven Commands — Must Remember

Command	Use
`mvn -v`	Check Maven version.
`mvn clean`	Delete previous build files.
`mvn compile`	Compile main Java code.
`mvn test`	Run test cases.
`mvn clean test`	Clean files and run tests.
`mvn package`	Create a JAR/WAR package.
`mvn install`	Install the built project in the local Maven repository.
`mvn dependency:tree`	Show project dependencies.
`mvn -Dtest=DiscountCalculatorTest test`	Run one test class.

JUnit 5 — Annotations

- `@Test` — Marks a test method.
- `@BeforeEach` — Runs before every test.
- `@AfterEach` — Runs after every test.
- `@BeforeAll` — Runs once before all tests.
- `@AfterAll` — Runs once after all tests.
- `@Disabled` — Disables a test.
- `@ParameterizedTest` — Runs a test with multiple input values.

JUnit 5 — Assertions

- `assertEquals(expected, actual)` — Checks equality.
- `assertTrue(condition)` — Checks whether true.
- `assertFalse(condition)` — Checks whether false.
- `assertNull(value)` — Checks for `null`.
- `assertNotNull(value)` — Checks for non-null.
- `assertThrows(Exception.class, ...)` — Checks whether an exception is thrown.

Interview priority: Remember `mvn test`, `mvn clean`, `@Test`, `@BeforeEach`, `@AfterEach`, `assertEquals()`, and `assertThrows()`.



Follow this 4-step development and testing process:

Step 1: Understand and Design Understand the problem statement, identify inputs, expected outputs, constraints, and design the solution.

Step 2: Write Pseudocode Write the solution logic in simple, step-by-step language before coding.

Step 3: Write Code and Test Cases Implement the solution in Java, then write JUnit test cases for normal inputs, edge cases, and invalid inputs.

Step 4: Run the Tests Run the test cases using `mvn test`, check the results, fix any failures, and rerun the tests.

Remember: Understand → Pseudocode → Code + Tests → Run Tests.

JUnit 5 Architecture — Explain This Diagram in an Interview

Interview answer:

"JUnit 5 architecture has three main components: JUnit Platform, JUnit Jupiter, and JUnit Vintage. It also supports extensions for customized testing.

1. JUnit Jupiter API: Provides annotations such as `@Test`, `@BeforeEach`, and assertions to write JUnit 5 tests.
2. JUnit Vintage: Allows older JUnit 3 and JUnit 4 tests to run on the JUnit Platform.
3. Extensions: Allow developers to customize test behaviour, such as setting up test data or managing resources.
4. JUnit Platform: Provides the foundation to discover and launch tests through test engines.
5. IDEs and tools: Tools such as IntelliJ IDEA, Eclipse, and Maven can launch tests through the Platform."

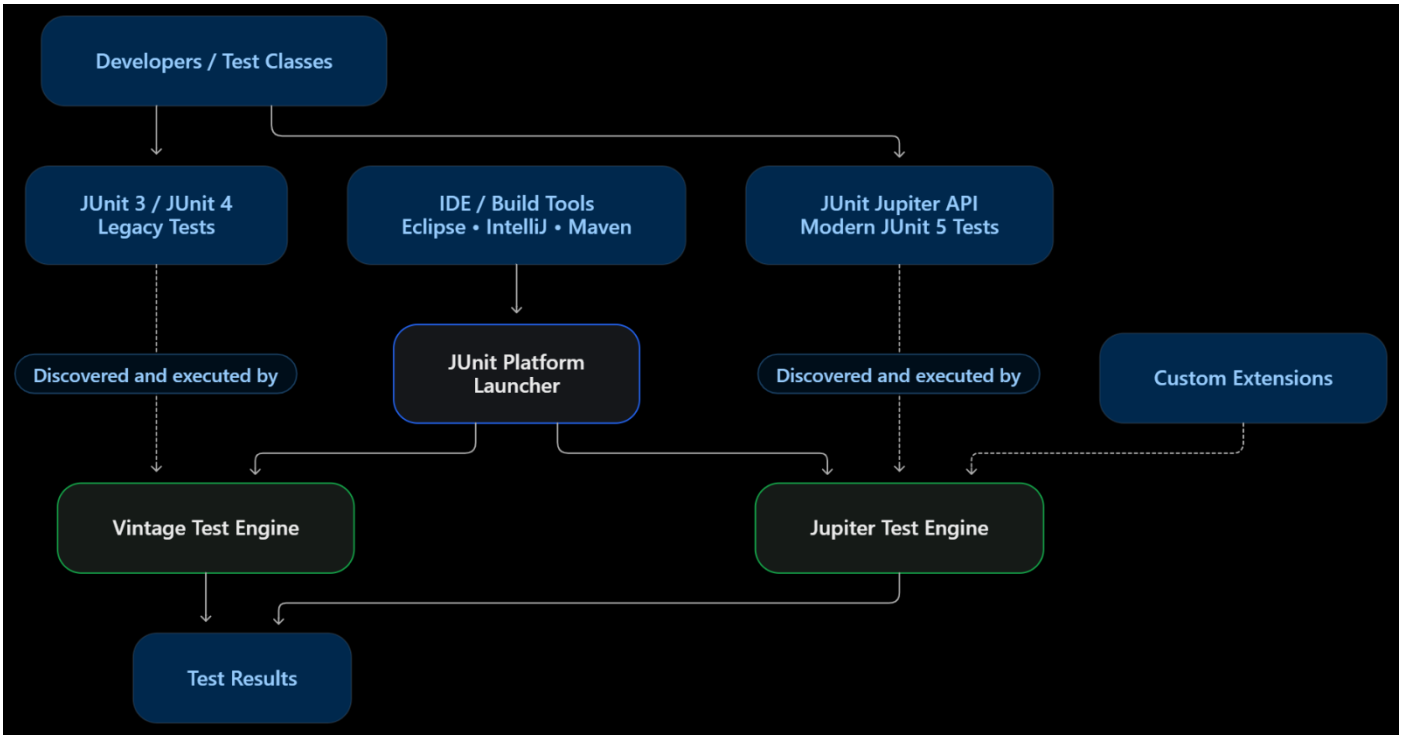
How the diagram works

Developers write tests using Jupiter or legacy JUnit APIs → the appropriate test engine runs them through the JUnit Platform → IDEs and build tools can launch the tests and display the results.

Remember for interviews: Jupiter = modern JUnit tests; Vintage = legacy tests; Platform = test execution foundation.

Remember for your interview:

- Jupiter API: Write modern JUnit 5 tests.
- Vintage Engine: Run legacy JUnit 3 and JUnit 4 tests.
- Platform: Provides the foundation for launching and coordinating tests.
- IDE/Maven: Launch tests and display results.
- Extensions: Customize test execution, especially with Jupiter.



JUnit 5 Architecture — Cinema Analogy 🎬

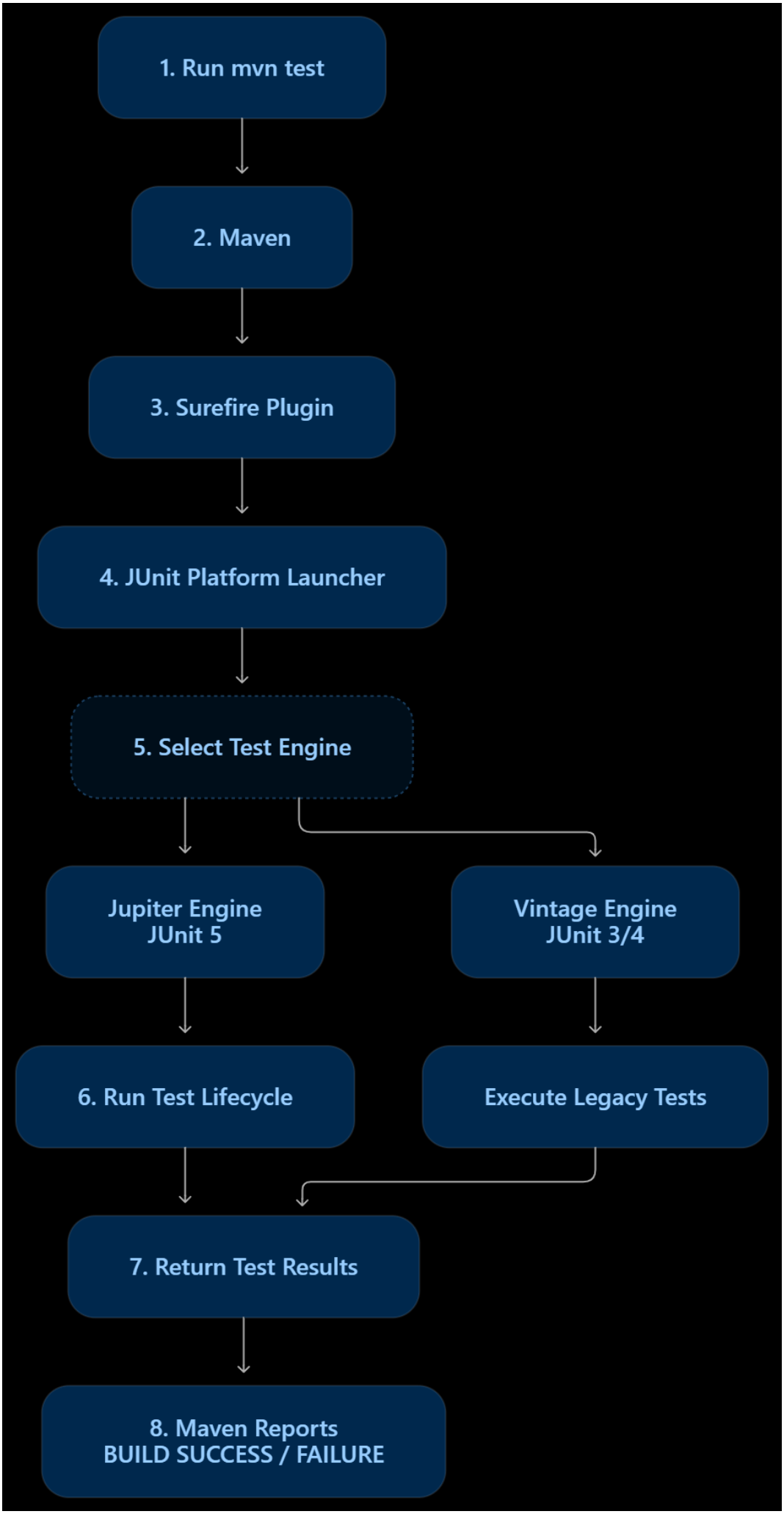
“Imagine JUnit 5 as a cinema.”

- JUnit Platform = Cinema building: It provides the common infrastructure to run different types of tests.
- IDE/Maven = Ticket counter: You use it to request and launch tests.
- JUnit Jupiter = Modern movie system: It handles modern JUnit 5 tests.
- JUnit Vintage = Old movie system: It allows older JUnit 3 and JUnit 4 tests to run in the same cinema.
- Test Engine = Movie player: It knows how to discover and execute its type of tests.

The main idea: The cinema building supports different movie systems. Similarly, the JUnit Platform supports different test engines and delegates test execution to the appropriate engine.

Remember: Platform = building, Maven/IDE = ticket counter, Jupiter = modern tests, Vintage = legacy tests, Test Engine = movie player.

JUnit 5: `mvn test` — Each Step Explained


1. Run mvn test
2. Maven
3. Surefire Plugin
4. JUnit Platform Launcher
5. Select Test Engine
Jupiter Engine
JUnit 5
Vintage Engine
JUnit 3/4
6. Run Test Lifecycle
Execute Legacy Tests
7. Return Test Results
8. Maven Reports
BUILD SUCCESS / FAILURE

Each step: simple meaning + interview answer

1. `mvn test` — Command

- Understand: You tell Maven to run the project's tests.
- Interview: “I execute `mvn test` from the directory containing `pom.xml` to build and run the configured tests.”

2. Maven — Build tool

- Understand: Reads `pom.xml`, manages dependencies, and runs the build lifecycle up to the test phase.
- Interview: “Maven manages the project build and coordinates compilation and test execution.”

3. Maven Surefire Plugin — Test runner integration

- Understand: Finds eligible test classes and arranges for them to run.
- Interview: “Surefire integrates test execution into Maven's test phase and reports the results.”

4. JUnit Platform Launcher — Test launcher

- Understand: Coordinates test discovery and execution through compatible test engines.
- Interview: “The JUnit Platform provides the infrastructure for launching tests using the appropriate test engine.”

5. Test Engine — Selects how tests run

- Jupiter Engine: Executes modern JUnit 5 tests.
- Vintage Engine: Executes legacy JUnit 3 and JUnit 4 tests when configured.
- Interview: “The engine understands the test framework and executes its tests.”

6. Test Lifecycle — Executes each test

For Jupiter tests, the lifecycle is:

`@BeforeAll` → `@BeforeEach` → `@Test` → `@AfterEach` → `@AfterAll`

- Understand: Prepare, run, and clean up tests.
- Interview: “Jupiter invokes lifecycle methods around the test methods so that setup and cleanup happen at the right time.”

7. Test Results — Pass or fail

- Understand: JUnit records whether assertions passed or tests failed.
- Interview: “The test engine returns execution results to the test runner.”

8. Maven Report — Final status

- Understand: Maven displays the build result and test summary.
- Interview: “Maven reports `BUILD SUCCESS` when the build succeeds, or `BUILD FAILURE` when a failure causes the build to fail.”

Remember: `BUILD SUCCESS` does not necessarily mean every test passed if the build has been configured to tolerate test failures.

Complete answer to speak in the interview

“When I run `mvn test`, Maven reads `pom.xml` and executes the build lifecycle up to the test phase. The Surefire Plugin discovers the test classes and integrates with JUnit to execute them. The JUnit Platform coordinates test execution through the appropriate engine. The Jupiter Engine runs JUnit 5 tests, while Vintage can run older JUnit tests if configured. The engine executes the tests and their lifecycle methods, and finally Maven displays the test results and build status.”